

JavaScript - Day 5

Anonymous Functions, Arrow Functions, Callbacks & Array Iteration Methods

Student Notes & Interview Preparation Guide

Topics covered:

Anonymous Functions | Arrow Functions | Lexical this
Callback Functions | Sync vs Async | Callback Hell
forEach | find | some | sort | map | filter | every | reduce

Topics Covered

1. Anonymous Functions
2. Arrow Functions
3. Lexical this
4. Callback Functions
5. Synchronous vs Asynchronous JavaScript
6. Callback Hell
7. Array Iteration Methods: forEach, find, some, sort, map, filter, every, reduce

1. Anonymous Functions

Why Do We Need Them?

Sometimes we don't need to create a separate named function. We only want to store a function inside a variable or pass it as an argument. In such cases we use Anonymous Functions.

Normal Function vs Anonymous Function

Normal Function

```
function greet() {  
    console.log("Hello");  
}  
  
greet();
```

Anonymous Function

```
let greet = function() {  
    console.log("Hello");  
};  
  
greet();
```

Characteristics

- ✓ No function name
- ✓ Usually stored in a variable
- ✓ Can be passed as an argument
- ✓ Commonly used with callbacks

Syntax

```
let variableName = function() {  
  // code  
}
```

Real World Example

```
button.addEventListener("click", function() {  
  console.log("Button Clicked");  
});
```

The function only runs when the button is clicked, so giving it a name is unnecessary.

Common Beginner Mistake

✗ **Wrong** — Anonymous functions cannot exist alone

```
function() {  
  console.log("Hello");  
}
```

✓ **Correct**

```
let greet = function() {  
  console.log("Hello");  
}
```

2. Arrow Functions

Why Were Arrow Functions Introduced?

Writing function expressions repeatedly becomes lengthy. ES6 introduced Arrow Functions to write functions in a shorter and cleaner way.

Function Expression vs Arrow Function

Function Expression

```
let multiply = function(a,b,c){  
  return a*b*c;  
}
```

Arrow Function

```
let multiply = (a,b,c) => a*b*c;
```

General Syntax

Case	Syntax
Multiple parameters	(a,b) => a+b
Single parameter	a => a*a
No parameters	() => console.log("Hello")

Arrow Function Rules

Rule 1 — Single Line Return

If there is only one statement, JavaScript automatically returns the value.

```
let add = (a,b) => a+b;
```

Equivalent to:

```
let add = function(a,b){
  return a+b;
}
```

Rule 2 — One Parameter

Parentheses are optional with a single parameter.

```
num => num*num

(num) => num*num
```

Both are valid.

Rule 3 — No Parameters

Parentheses are mandatory.

```
() => console.log("Hello")
```

Common Mistake: Forgetting Return

✗ Wrong — missing return inside block body

```
let square = num => {
  num * num;
}

console.log(square(5));
```

Output:

```
undefined
```

✓ Correct — use implicit return OR explicit return

```
let square = num => num*num;
```

or

```
let square = num => {  
  return num*num;  
}
```

Arrow Function Returning an Object

Note: This is one of the most common interview questions.

✗ Wrong

```
let user = (name,age) => {  
  name,  
  age  
}
```

Output:

undefined

✓ Correct — wrap the object in parentheses

```
let user = (name,age) => ({  
  name,  
  age  
});
```

Output:

```
{ name:"Yathe", age:25 }
```

JavaScript treats `{ }` as a function block. Wrapping the object in `( )` tells JavaScript it is an object.

3. Lexical this in Arrow Functions

Normal Function this

```
const obj = {  
  name:"JS",  
  
  normal(){  
    console.log(this.name);  
  }  
};  
  
obj.normal();
```

Output:

JS

Arrow Function this

```
const obj = {  
  name: "JS",  
  
  arrow: () => {  
    console.log(this.name);  
  }  
};
```

Output:

undefined

Why?

Arrow functions do not create their own 'this'. They inherit 'this' from the surrounding scope. This behavior is called Lexical This.

Comparison Table

Normal Function	Arrow Function
Has its own this	No own this
Depends on the caller	Uses surrounding scope
Useful in objects	Useful in callbacks

4. Callback Functions

What is a Callback?

A callback is a function passed as an argument to another function and executed later.

Why Do We Need Callbacks?

Suppose Task 2 should execute only after Task 1 finishes. Callbacks help control the execution order.

Example

```
function task1(callback){  
  console.log("Task 1");  
  callback();  
}
```

```
function task2(){
  console.log("Task 2");
}

task1(task2);
```

Output:

```
Task 1
Task 2
```

Real World Analogy

```
Order Food
↓
Food Prepared
↓
Waiter Calls You
↓
You Collect Food
```

The waiter calling you is similar to a callback.

Four Ways to Write Callbacks

1. Using Function Declaration

```
function greeting(name){ console.log(`Hello ${name}`); }

function reception(callback){ callback("JavaScript"); }

reception(greeting);
```

2. Using Function Expression

```
let greeting = function(name){ console.log(`Hello ${name}`); }

reception(greeting);
```

3. Inline (Anonymous) Callback

```
reception(function(name){
  console.log(`Hello ${name}`);
});
```

4. Arrow Function Callback

```
reception(name => {
  console.log(`Hello ${name}`);
});
```

Important Difference

Syntax	Meaning
task1(task2)	Give task2 as a reference — runs later
task1(task2())	Run task2 immediately, pass result

5. Synchronous vs Asynchronous JavaScript

Synchronous Execution

JavaScript executes code line by line.

```
console.log("Start");
console.log("End");
```

Output:

```
Start
End
```

Asynchronous Execution

```
console.log("Start");

setTimeout(()=>{
  console.log("Done");
},5000);

console.log("End");
```

Output:

```
Start
End
Done
```

setTimeout() does not stop JavaScript execution. It schedules the callback and allows JavaScript to continue running.

Real World Analogy

```
Order Pizza
↓
Pizza Preparing
↓
Meanwhile Watch TV
↓
Pizza Arrives Later
```

This is asynchronous behavior.

setTimeout Syntax

```
setTimeout(callbackFunction, timeInMilliseconds)
```

Example:

```
setTimeout(()=>{  
  console.log("Hello");  
},2000);
```

6. Callback Hell

What is Callback Hell?

When callbacks become deeply nested.

```
step1(()=>{  
  step2(()=>{  
    step3(()=>{  
      console.log("Done");  
    });  
  });  
});
```

Problems

- ✗ Hard to read
- ✗ Hard to debug
- ✗ Hard to maintain

Names

This pyramid structure is known as:

- Callback Hell
- Pyramid of Doom

Note: Promises and async/await were introduced in modern JavaScript to solve Callback Hell.

Calculator Using Callback

```
function calculate(a,b,operation){  
  return operation(a,b);  
}
```

```
let add = calculate(10,5,(a,b)=>a+b);
let sub = calculate(10,5,(a,b)=>a-b);
let mul = calculate(10,5,(a,b)=>a*b);
```

Benefit

Same function can perform multiple operations. Reusable and flexible.

8. Array Iteration Methods

Why Are They Needed?

Instead of writing loops repeatedly:

```
for(let i=0;i<arr.length;i++){
  ...
}
```

JavaScript provides built-in methods that automatically iterate through arrays.

Overview Table

Method	Purpose	Returns
forEach()	Iterate over each element	undefined
find()	First element matching condition	Element or undefined
some()	At least one element matches?	Boolean
sort()	Sort elements	Sorted array (in-place)
map()	Transform every element	New array
filter()	Get all matching elements	New array
every()	All elements match?	Boolean
reduce()	Reduce array to single value	Single value

forEach()

Purpose

Runs a callback for every element. Does not return a new array.

Syntax

```
arr.forEach((element, index, array) => { });
```

Example

```
let arr = [1,2,3];

arr.forEach(ele => {
  console.log(ele);
});
```

Output:

```
1
2
3
```

Parameters

Parameter	Required
element	Yes
index	Optional
array	Optional

Return Value

```
let result = arr.forEach(ele => console.log(ele));
console.log(result);
```

Output:

```
undefined
```

Use Cases

- Printing data
- Updating DOM
- Processing API responses
- Logging

find()

Purpose

Returns the first element that satisfies the condition. Stops searching after first match.

Example

```
let arr = [1,2,3,4,5];  
  
let result = arr.find(ele => ele%2===0);  
  
console.log(result);
```

Output:

2

some()

Purpose

Checks whether at least one element satisfies the condition. Returns Boolean.

Example

```
let arr = [1,3,5,8];  
  
let result = arr.some(ele => ele%2===0);  
  
console.log(result);
```

Output:

true

find() vs some()

find()	some()
Returns the matching element	Returns true or false
Gives you the actual value	Tells you if it exists
Example: 8	Example: true

When To Use?

- Need the element itself? → Use find()
- Need yes/no answer? → Use some()

sort()

Purpose

Sorts array elements. Modifies the original array.

String Sorting

```
let arr = ["React","JavaScript","MERN"];

console.log(arr.sort());
```

Number Sorting Problem

```
let arr = [10,2,30,4];

console.log(arr.sort());
```

Output:

```
[10, 2, 30, 4] ← Unexpected!
```

sort() converts values to strings first and sorts in dictionary (alphabetical) order.

Fix Using Comparator Function

Ascending

```
arr.sort((a,b) => a-b);
```

Descending

```
arr.sort((a,b) => b-a);
```

Note: sort() modifies the original array. If you want to preserve the original, copy it first: [...arr].sort(...)

map()

Purpose

Transforms every element and returns a new array. The original array remains unchanged.

Example

```
let arr = [1,2,3];  
  
let result = arr.map(ele => ele*2);  
  
console.log(result);
```

Output:

```
[2, 4, 6]
```

map() vs forEach()

map()	forEach()
Returns a new array	Returns undefined
Used for transformation	Used for side effects
Most commonly used in React	Less used for rendering
Can be chained	Cannot be chained

filter()

Purpose

Returns a new array with all elements that satisfy the condition.

Example

```
let arr = [1,2,3,4,5,6];  
  
let result = arr.filter(ele => ele%2===0);  
  
console.log(result);
```

Output:

```
[2, 4, 6]
```

every()

Purpose

Checks whether ALL elements satisfy the condition. Returns Boolean.

Example

```
let arr = [10,20,30];

let result = arr.every(ele => ele > 5);

console.log(result);
```

Output:

true

filter() vs every()

filter()	every()
Returns matching elements	Returns true or false
Gives an array	Gives a validation result

some() vs every() — Important Distinction

some()	every()
At least ONE element must match	ALL elements must match
If any element passes → true	If any element fails → false

reduce()

Purpose

Reduces an entire array into a single value.

Real World Example

Shopping cart total:

[1000, 500, 200] → 1700

Syntax

```
array.reduce(
  (accumulator, currentValue) => {
    return updatedValue;
  },
  initialValue
);
```

Example 1 — Sum of Prices

```
let prices = [1000,1200,800];

let total = prices.reduce(
  (sum,price) => sum+price
);

console.log(total);
```

Output:

```
3000
```

Example 2 — Total String Length

```
let words = ["JS","React","MERN"];

let totalLength = words.reduce(
  (total,word) => total+word.length,
  0
);

console.log(totalLength);
```

Parameters Explained

Parameter	Meaning
accumulator	Stores the result of the previous iteration
currentValue	The current element being processed
index	Current element's index (optional)
array	Original array (optional)
initialValue	Starting value for the accumulator

Summary Table — All Array Methods

Method	Returns	Changes Original?	Common Use
forEach()	undefined	No	Printing, logging
find()	First match / undefined	No	Search for one element

Method	Returns	Changes Original?	Common Use
some()	Boolean	No	Check existence
sort()	Sorted array	Yes	Sorting data
map()	New array	No	Transform elements
filter()	New array	No	Get matching elements
every()	Boolean	No	Validation
reduce()	Single value	No	Totals, aggregation

Interview Questions

Q1. Difference between Function Expression and Arrow Function?

Arrow functions provide shorter syntax and do not have their own 'this'. Function expressions have their own 'this' and are more verbose.

Q2. What is a Callback Function?

A function passed as an argument to another function and executed later.

Q3. Difference between Synchronous and Asynchronous JavaScript?

Synchronous	Asynchronous
Executes line by line	Schedules tasks for later
Blocks next line until done	Does not block execution
Example: normal code	Example: setTimeout, API calls

Q4. Difference between find() and filter()?

find()	filter()
Returns first matching element	Returns all matching elements
Stops after first match	Scans the entire array
Returns value or undefined	Always returns an array

Q5. Difference between some() and every()?

some() checks if at least one element matches. every() checks if all elements match.

Q6. Why is map() preferred over forEach() in React?

Because map() returns a new array which can be directly rendered as JSX. forEach() returns undefined and cannot be used for rendering.

Q7. What is Callback Hell?

When callbacks become deeply nested, forming a pyramid structure that is hard to read, debug, and maintain. Also known as the Pyramid of Doom.

Q8. Does sort() modify the original array?

Yes. sort() sorts in-place and modifies the original array. To preserve the original, create a copy first: [...arr].sort()

Q9. What is the difference between Passing and Calling a function as a callback?

task1(task2)	task1(task2())
Passes the reference	Calls immediately
task2 runs later	task2 runs now
Correct for callbacks	Passes return value instead

Q10. What is Lexical this?

Arrow functions inherit 'this' from the surrounding scope instead of creating their own. This is called Lexical this.

Key Takeaways

- ✓ Anonymous functions have no name and are stored in variables.
- ✓ Arrow functions provide shorter syntax.
- ✓ Arrow functions use lexical this — they do not have their own this.
- ✓ Callbacks execute after another function finishes.
- ✓ JavaScript can perform asynchronous operations using callbacks.
- ✓ Callback Hell creates deeply nested, unreadable code.
- ✓ forEach() is for iteration — returns undefined.
- ✓ map() transforms data — returns a new array.

- ✓ `filter()` returns all matching elements as a new array.
- ✓ `find()` returns the first matching element only.
- ✓ `some()` checks if at least one element matches.
- ✓ `every()` checks if all elements match.
- ✓ `reduce()` converts an array into a single value.
- ✓ `sort()` modifies the original array. Use a comparator for numbers.